

مینی پروژه سوم

درس:

یادگیری ماشین

نیمسال اول تحصیلی

۱۴۰۴-۱۴۰۵

تاریخ تحویل

۱۴۰۴/۱۰/۱۲

امید طالبی

گام اول – بارگذاری مجموعه داده و تحلیل اولیه.....	۳
گام دوم – پیش پردازش.....	۶
گام سوم – پیاده سازی الگوریتم knn.....	۱۰
گام چهارم – درخت تصمیم.....	۱۶
گام پنجم – پیاده سازی مدل LDA.....	۲۰
گام ششم – جمع بندی و مقایسه نهایی.....	۲۲

❖ شرح مراحل انجام پروژه

گام اول – بارگذاری مجموعه داده و تحلیل اولیه

این مجموعه داده (Forest cover type) شامل اطلاعات توپوگرافی و محیطی مناطق مختلف جنگلی است که هدفش پیش بینی نوع پوشش جنگلی بر اساس ویژگی های ورودی میباشد. این مجموعه داده یکی از داده های شناخته شده در مسائل طبقه بندی چند کلاسه است و به دلیل حجم بسیار بالا و نامتوازن بودن کلاس ها چالش برانگیز خواهد بود.

پس از بارگذاری کردن داده ها به این رسیدیم که :

dataset shape is: (581012, 55)

تعداد رکوردها ۵۸۱۰۱۲ تا و ۵۵ تا ویژگی داریم که از ۵۵ تا ۱۰ تا عددی هستند و یه کلاس تارگت میباشد و مابقی که ۴۴ باینری (boolean) میباشد.

number binary features : 44

number binary features : 10

البته تعداد کلاس های تارگت و تعداد نمونه ها به صورت زیر میباشد (که مشاهده میکنیم که داده ها به صورت inbalance هستند)

Class2: 283301

Class1: 211840

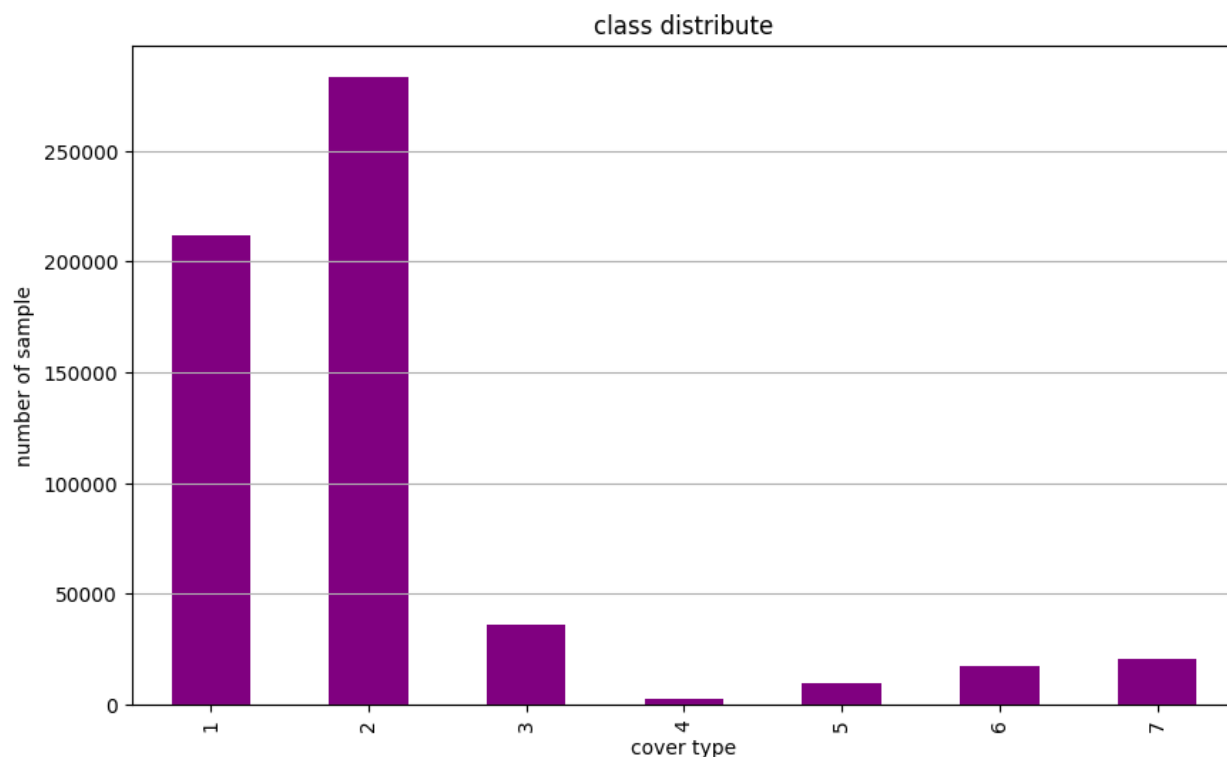
Class3: 35754

Class7: 20510

Class6: 17367

Class5: 9493

Class4: 2747



که از این ویژگی های عددی که ۱۰ تا بود ستون های زیر را شامل میشود:

Elevation ,Aspect, Slope, Horizontal_Distance_To_Hydrology, Vertical_Distance_To_Hydrology, Horizontal_Distance_To_Roadways, Hillshade_۹am, Hillshade_Noon,Hillshade_۳pm, Horizontal_Distance_To_Fire_Points

این ویژگی ها دارای مقیاس های بسیار زیاد هست به همین دلیل اینها رو باید استاندارد کنیم.

ما بقی ۴۴ تا ویژگی باینری که به صورت ۰ و ۱ نمایش داده شدند و نشان دهنده حضور یا عدم حضور یک ویژگی خاص هستند. این دسته از ویژگی ها نیاز به استاندارد سازی ندارند و مستقیماً قابل استفاده در مدل ها هستند.

و در انتها به طبق صورت سوال که گفته به میانگین و یه سری اطلاعات آماری پردازیم با استفاده از دستور info و describe یه سری اطلاعات آماری گرفته شده که میتوان در کد مشاهده نمود.

۱- با توجه به imbalanced بودن مجموعه داده، استفاده از معیار ارزیابی accuracy چه مشکلی ایجاد

میکند؟ راه جایگزین چیست؟

در مجموعه داده‌های نامتوازن، استفاده از معیار Accuracy دقیق نمیباشد. چون اگر یک یا چند کلاس سهم خیلی بیشتری از داده داشته باشند، مدلی که فقط همان کلاس اکثریت را پیش‌بینی کند هم می‌تواند Accuracy بالایی بگیرد، در حالی که در تشخیص کلاس‌های کم عملاً عملکردی ندارد. یعنی accuracy فقط تعداد پیش‌بینی‌های درست را می‌شمارد، اما به این توجه نمی‌کند که این درستی مربوط به کدام کلاس‌هاست. به همین دلیل در چنین داده‌هایی معیار مناسبی نیست.

در عوضش باید از معیارهای دیگر مثل Precision , Recall, F1-score , Confusion Matrix استفاده کرد.

۲- ویژگی‌های عددی و باینری چه سهمی از داده را تشکیل داده اند و چه معنایی دارند؟

در این دیتاست داده‌های عددی شامل ۱۰ ستون و ستون‌های غیر ۴۴ تا میباشد

پس :

داده‌های عددی ۱۰ به کلش که $10 + 44$ تا میباشد برابر میشود با ۰.۱۸

و برای داده‌ها باینری برابر میشود با ۰.۸۱

که ویژگی‌های عددی بیانگر اطلاعات پیوسته و فیزیکی محیط هستند مثل ارتفاع ، شیب زمین و.. که این ویژگی‌ها مستقیماً روی محاسبه فاصله تاثیر میگذارد.

باقی ویژگی‌ها به سری اطلاعات باینری میباشد که بیانگر وجود یا عدم وجود ویژگی میباشد مثلاً نوع خاک که به صورت One_Hot_encoding نمایش داده میشود.

۳- اقدام ویژگی‌ها رفتار متفاوتی از نظر پراکندگی یا مقیاس دارند؟

در بین ویژگی‌های عددی بعضی از ویژگی‌ها در order هزارتا بودن یعنی مقادیرشون ۱۰۰۰ یا ۳۰۰۰ بود ولی بعضی هاشون در حدود ۱۰۰ تا بود برای مثال Horizontal Distance و Horizontal Distance to Roadways to Fire Points مقیاس بزرگتری نسبت به بقیه داشتن که اگر این ویژگی‌ها بدون استاندارد سازی وارد مدل شود، میتواند روی محاسبه فاصله تاثیر زیادی بگذارد و باعث این میشود که مدل به سایر ویژگی‌ها توجه کمتری داشته باشد. که این استانداردسازی برای مدل‌هایی مثل k-NN و LDA ضروری است.

گام دوم – پیش پردازش

با استفاده از دستور `isnull.sum()` متوجه این امر شدیم که در دیتاست مون داده ای نال وجود ندارد.

#	Column	Non-Null Count	Dtype
0	Elevation	581012 non-null	int64
1	Aspect	581012 non-null	int64
2	Slope	581012 non-null	int64
3	Horizontal_Distance_To_Hydrology	581012 non-null	int64
4	Vertical_Distance_To_Hydrology	581012 non-null	int64
5	Horizontal_Distance_To_Roadways	581012 non-null	int64
6	Hillshade_9am	581012 non-null	int64
7	Hillshade_Noon	581012 non-null	int64
8	Hillshade_3pm	581012 non-null	int64
9	Horizontal_Distance_To_Fire_Points	581012 non-null	int64
10	Wilderness_Area1	581012 non-null	int64
11	Wilderness_Area2	581012 non-null	int64
12	Wilderness_Area3	581012 non-null	int64
13	Wilderness_Area4	581012 non-null	int64
14	Soil_Type1	581012 non-null	int64
15	Soil_Type2	581012 non-null	int64
16	Soil_Type3	581012 non-null	int64
17	Soil_Type4	581012 non-null	int64
18	Soil_Type5	581012 non-null	int64
19	Soil_Type6	581012 non-null	int64
..			
53	Soil_Type40	581012 non-null	int64
54	Cover_Type	581012 non-null	int64

ولی اگر داده ای نال وجود داشت میتونیم روش های زیر را انجام بدیم :

۱ – Missing Completely at Random (MCAR)

در این مورد از دست رفتن داده کاملاً تصادفی بوده و به مقدار خودش و ستون های دیگر و به گروه خاص ارتباط ندارد. برای مثال بعد از اینکه ما با اپراتور با تلفن صحبت میکنیم ، اپراتور میگه در صورت تمایل به پاسخگویی نمره ۱ تا ۵ بدهید که تماسی که من برقرار میکنم و امتیازی که نمیدم هیچ ارتباطی با تماس دیگر ندارد.

۲ – Missing at Random (MAR)

گم شدن داده به خودش وابسته نیست و به ستون یا ویژگی های دیگر در دیتاست وابسته است. برای مثال ممکن است در بعضی از فرم ها خانم ها سن یا وزنشان را کمتر از آقایون وارد کنند.

۳ – Missing Not at Random (MNAR)

در این حالت گم شدن دیتا به این دلیل است که فردی که میخواهد اطلاعات را وارد کند چون مقدار آن خیلی زیاد یا خیلی کم یا خاص هست از وارد کردن آن جلوگیری میکند.

برای مثال میخواهد فرد وام بگیرد ولی داخل اطلاعات ، وضعیت خانه خود (استیجاری ، صاحب خانه) را پر نمیکند.

در ادامه راه حل های و راهکارهای پر کردن دیتا برای missing value را خواهیم داشت :

۱- میتوان با استفاده از mean , median مقادیر، نال رو پر کرد.

۲- در صورتی که داده به صورت categorical از مد استفاده کنیم.

۳- از متد Forward Fill and Backward Fill استفاده کنیم.

Forward fill

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	0.8
2	2021-01-03	apple	0.8
3	2021-01-04	apple	1.2
4	2021-01-01	mango	1.2
5	2021-01-02	mango	3.1
6	2021-01-03	mango	3.1
7	2021-01-04	mango	2.8

Backward fill



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	1.2
2	2021-01-03	apple	1.2
3	2021-01-04	apple	1.2
4	2021-01-01	mango	3.1
5	2021-01-02	mango	3.1
6	2021-01-03	mango	2.8
7	2021-01-04	mango	2.8

البته میتوان از روش های آماری دیگر که به صورت متد هستند نیز استفاده کرد :

برای مثال MICE (Multiple Imputation) که روش بسیار خوبی هست و به صورت زیر عمل میکند :

۱- هر ستون دارای missing به صورت شرطی و چندباره تخمین زده می شود

۲- uncertainty (عدم قطعیت) را هم مدل می کند

۳- نتایج بسیار واقعی تر هستند

البته روش های دیگری هم وجود دارد.

۱-۲ چرا در این پروژه به مجموعه validation نیاز داریم؟ اگر فقط داده را به دو بخش train و test تقسیم

کنیم، چه مشکلی در تنظیم هایپر پارامترها و مقایسه مدل ها به وجود می آید؟

مجموعه validation برای تنظیم هایپر پارامترها و مقایسه مدل ها استفاده می شود. اگر داده ها فقط به train و test تقسیم شوند، مجبور می شویم از داده test برای انتخاب مدل یا تنظیم پارامترها استفاده کنیم که این کار باعث نشت اطلاعات (data leakage) میشود. پس میتوان گفت

train برای یادگیری مدل

validation برای انتخاب مدل و تنظیم پارامترها

test فقط برای ارزیابی نهایی

۲-۲ آیا مجموعه داده Forest Cover Type به استاندارد سازی نیاز دارد؟ توضیح دهید.

بله، این داده نیاز به استاندارد کردن دارد. چون ۱۰ تا از ویژگی ها مون عددی بودند و چون دارای مقیاس های بزرگ هستند. و در الگوریتم هایی مثل k-NN و LDA، ویژگی هایی با مقیاس بزرگ تر می توانند بر محاسبه فاصله یا مرز تصمیم گیری غلبه کنند و باعث این میشود مدل به سایر ویژگی ها توجه کافی نداشته باشد.

۳-۱۲ اگر استاندارد سازی فقط بر پایه داده train انجام نشود چه نوع خطایی رخ میدهد؟

اگر این اتفاق رخ بدهد در واقع داریم به مدل به طور دید میدیم و همین منجر به خروجی اشتباه شد چونکه در این صورت مدل خیلی خوب روی train و حتی روی test به خوبی عمل میکند ولی به محض اینکه وارد فضای production شود سبب میشود که دقت به شدت بیاد پایین و کارایی مناسبی نخواهد داشت.

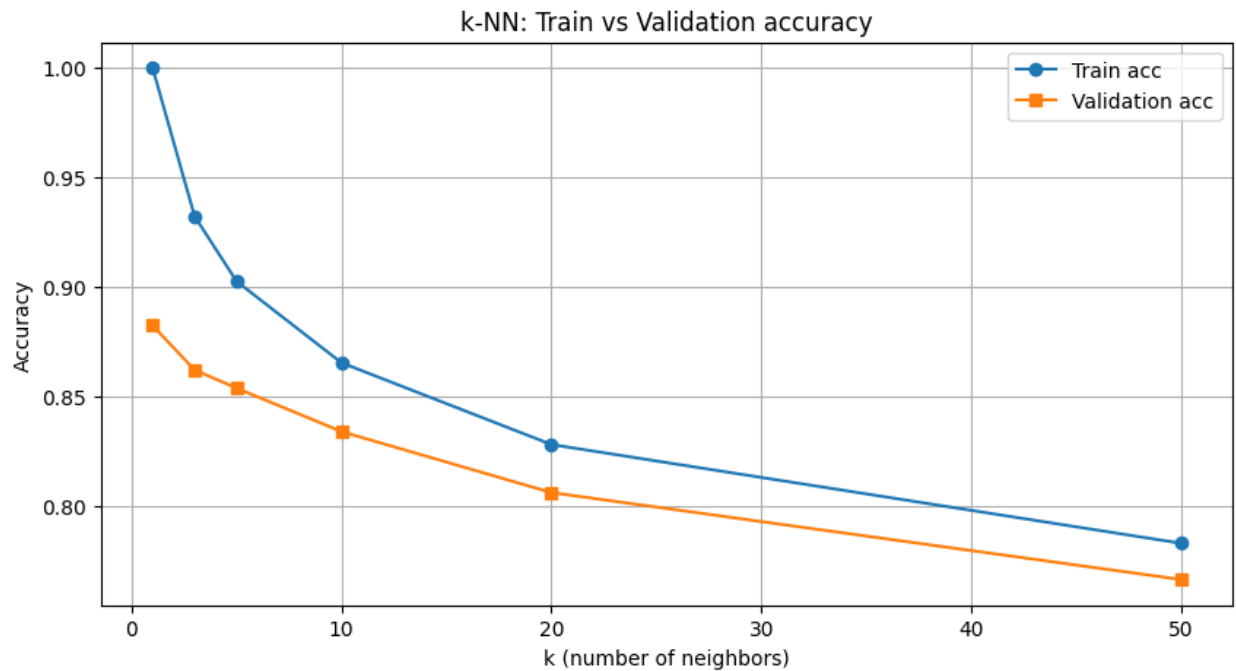
در واقع عملکرد مدل به صورت مصنوعی بهتر از واقعیت دیده می شود

ارزیابی نهایی غیر واقعی و خوش بینانه خواهد بود

مدل در دنیای واقعی عملکرد ضعیف تری خواهد داشت

به همین دلیل، استاندارد سازی باید فقط روی داده train انجام شود و سپس همان تبدیل روی validation و test اعمال گردد.

گام سوم – پیاده سازی الگوریتم K_NN



رفتار مدل با تغییر k

برای مقادیر کوچک: دقت روی داده های آموزشی بسیار بالا است. و مدل تقریباً داده های آموزشی را حفظ می کند.
با افزایش مقدار k دقت در آموزشی کم می شود و مدل ساده تر می شود و جزئیات محلی را کمتر در نظر می گیرد.

رفتار روی داده های validation

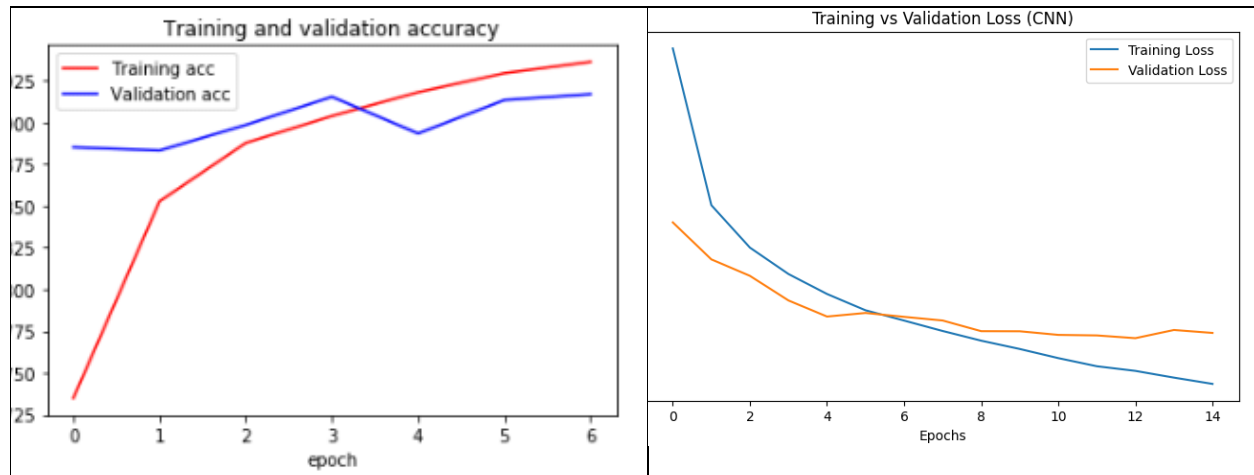
برای k های کوچک، دقت validation پایین تر از train است

با افزایش k، دقت validation ابتدا افزایش می یابد (البته در مدل ما این اتفاق نیوفتاده است)

در مقادیر بزرگ k، دقت validation دوباره کاهش پیدا می کند

این رفتار نشان دهنده وجود یک مقدار k بهینه است که تعادل مناسبی بین بیش برآزش و کم برآزش ایجاد می کند.

برای مثال در تصویر زیر میتوان این توضیحات رو مشاهده نمود:



تحلیل bias و variance با تغییر k

k = 1

variance بسیار بالا

bias بسیار کم

مدل به نویز داده حساس است (overfitting)

k های کوچک (۳ و ۵)

variance هنوز نسبتاً بالا

bias کم

مدل هنوز به ساختار محلی داده حساس است

k های متوسط (۱۰ و ۲۰)

تعادل مناسب بین bias و variance

بهترین عملکرد روی validation

این محدوده معمولاً بهترین انتخاب برای مدل نهایی است

k های بزرگ (50)

bias بالا

variance کم

مدل بیش از حد ساده شده و ساختار واقعی داده را خوب یاد نمی گیرد (underfitting)

انتخاب بهترین مقدار k

بهترین مقدار k بر اساس بیشترین validation accuracy انتخاب شد، نه بر اساس train accuracy. این کار باعث می شود مدل قابلیت تعمیم به داده های دیده نشده را حفظ کند. که البته چون در مدل ما همچنین حالت رخ نداده است من اون نقطه ای رو در نظر گرفتم که نه خیلی overfit و نه underfit باشد مثلاً نقاط میانی خوب میباشد و انجایی که فاصله بین داده آموزشی و ولیدیشن نیز کم باشد مثلاً ۵.

ارزیابی مدل نهایی روی داده test

برای ارزیابی نهایی مدل، از معیارهای زیر میتوانیم استفاده کنیم:

Confusion Matrix/ F1-score (Macro F1)/Recall / Precision / Accuracy

۱-۳ چرا NN-k یک روش غیر پارامتریک است و ظرفیت مدل چگونه با تغییر k دگرگون میشود؟

هیچ فرض مشخصی درباره شکل توزیع داده‌ها ندارد، مدلی از قبل آموزش نمی‌دهد، کل داده‌های train را نگه می‌دارد و تصمیم‌گیری را هنگام پیش‌بینی انجام می‌دهد و نکته مهتر این است که مدل کاملاً به k وابسته است

ویژگی	k کوچک (ظرفیت بالا)	k بزرگ (ظرفیت پایین)
مرز تصمیم‌گیری	پیچیده و نامنظم	ساده و هموار
حساسیت به نویز	بسیار حساس	مقاوم در برابر نویز
ریسک اصلی	بیش‌برازش (Overfitting)	کم‌برازش (Underfitting)
عملکرد با داده جدید	احتمالاً ضعیف	احتمالاً ضعیف

۲-۳ چه تفاوتی میان رفتار train accuracy و validation accuracy هنگام تغییر k مشاهده میشود؟

در k های کوچک، train accuracy بسیار بالا و validation accuracy پایین‌تر است

با افزایش k، فاصله بین train و validation کمتر می‌شود

در k بهینه، هر دو مقدار به هم نزدیک و نسبتاً بالا هستند

در k های بزرگ، هر دو دقت کاهش می‌یابند

این اختلاف رفتار ابزار اصلی برای تشخیص overfitting و underfitting است.

۳-۳ تغییر مقدار k چه تاثیری بر bias و variance دارد؟ برای تمامی مقادیر k، bias و variance را تحلیل کنید

مقدار k	وضعیت مدل	Variance	Bias
کوچک	Overfitting	زیاد	کم
متوسط	بهترین حالت	متعادل	متعادل
بزرگ	Underfitting	کم	زیاد

$k = 1$

Bias بسیار کم

چون مدل به شدت پیچیده و حساس است، می‌تواند تقریباً به تمام ویژگی‌های داده توجه کند و بنابراین bias بسیار کمی دارد.

Variance بسیار زیاد

مدل به طور کامل به داده‌های آموزشی حساس است و حتی نویزها و جزئیات کوچک را هم یاد می‌گیرد، که باعث overfitting می‌شود. این باعث variance بسیار بالا می‌شود.
نتیجه: مدل بسیار پیچیده است و قابلیت تعمیم بالایی ندارد.

$k = 3$

Bias کم

با افزایش k ، مدل دیگر به شدت تحت تأثیر نویز قرار نمی‌گیرد و شروع به یادگیری الگوهای کلی داده می‌کند.

Variance زیاد

مدل هنوز به جزئیات داده حساس است و برای داده‌های جدید هنوز به طور مناسبی عمل نمی‌کند.
نتیجه: هنوز مدل پیچیده است و امکان overfitting وجود دارد، اما بهبود یافته است.

$k = 5$

Bias متوسط

مدل از پیچیدگی کمتری برخوردار است و به خوبی قادر است به الگوهای عمومی داده توجه کند، اما هنوز احتمال خطا وجود دارد.

Variance متوسط

این مقدار k می‌تواند به خوبی ویژگی‌های اصلی داده را شبیه‌سازی کند، اما همچنان ممکن است به داده‌های نامتوازن یا نویز حساس باشد.
نتیجه: یک مقدار k مناسب برای مدل‌های پیچیده که هم bias و هم variance را کنترل می‌کند.

$k = 10$

Bias زیاد

با افزایش k ، مدل به طور بیشتری به الگوهای کلی داده توجه می کند و خطای بیشتری در یادگیری جزئیات داده ها به وجود می آید.

Variance کم

چون مدل به تعداد بیشتری از همسایگان توجه می کند، کمتر به تغییرات جزئی در داده حساس است. این منجر به **Variance کم** می شود.

نتیجه: مدل به طور کلی ساده تر است و احتمال **underfitting** بیشتر است.

$k = 20$

Bias بیشتر

مدل دیگر به شدت از پیچیدگی داده ها فاصله گرفته است و بسیاری از ویژگی های خاص و جزئیات داده ها را نمی تواند شبیه سازی کند.

Variance بسیار کم

مدل با افزایش k دیگر به داده ها حساس نیست و به طور کلی رفتار پایدارتری دارد.

نتیجه: مدل بسیار ساده و کم پیچیده است، با احتمال زیاد **underfitting**.

$k = 50$

Bias بسیار زیاد

با افزایش بیش از حد k ، مدل فقط می تواند الگوهای کلی و بزرگ تری را شبیه سازی کند و از هر گونه جزئیات خاص داده ها غافل می شود.

Variance بسیار کم

مدل کاملاً به داده های آموزشی وابسته نیست و رفتار بسیار پایداری از خود نشان می دهد.

نتیجه: مدل ساده ای است که ممکن است بسیاری از اطلاعات مهم داده ها را از دست بدهد و **underfitting** شدیدی ایجاد کند.

گام چهارم - درخت تصمیم

تحلیل Bias و Variance

عمق کم درخت max_depth کوچک:

Bias بالا: درخت تصمیم خیلی ساده می شود و قادر به یادگیری الگوهای پیچیده داده ها نیست.

Variance پایین: چون درخت به شدت ساده است، به داده های آموزشی خیلی حساس نیست.

عمق زیاد درخت max_depth بزرگ:

Bias پایین: درخت می تواند پیچیدگی های داده ها را یاد بگیرد و الگوهای جزئی را شبیه سازی کند

Variance بالا: درخت به داده های آموزشی حساس می شود و ممکن است در برابر داده های جدید دچار

overfitting شود.

تحلیل عملکرد مدل

برای مقادیر مختلف max_depth، رفتار مدل مورد بررسی قرار گرفت:

max_depth کوچک: احتمال underfitting وجود دارد، زیرا مدل نمی تواند پیچیدگی های داده ها را یاد بگیرد.

max_depth بزرگ: احتمال overfitting وجود دارد، زیرا درخت به داده های آموزشی بیش از حد حساس

می شود.

نمودار صحت و خطای طبقه بندی نادرست

در این بخش، نمودار صحت (Accuracy) و خطای طبقه بندی نادرست (Misclassification Error) برای مقادیر

مختلف max_depth رسم شد.

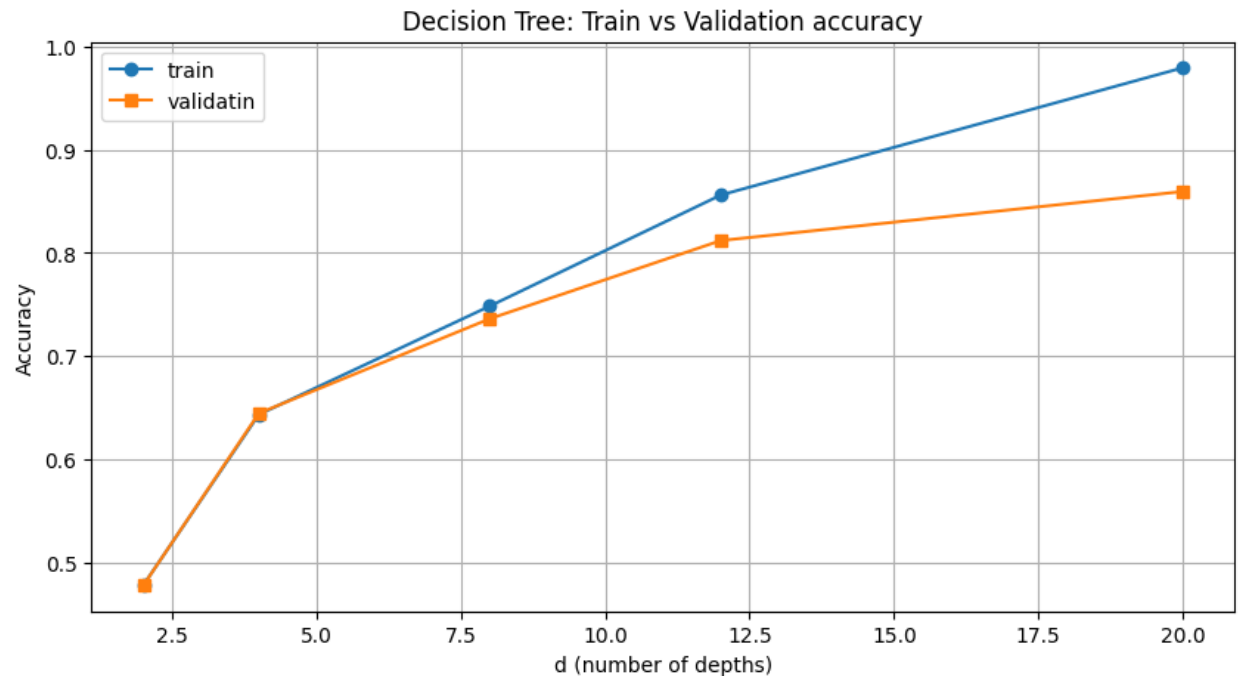
$$\text{Misclassification Error} = 1 - \text{Accuracy}$$

نمودار Accuracy نشان می دهد که با افزایش عمق درخت، ابتدا دقت مدل افزایش می یابد و سپس به ثبات می رسد یا

کاهش پیدا می کند.

نمودار Misclassification Error نشان می دهد که با افزایش عمق، خطای طبقه بندی نادرست ابتدا کاهش می یابد و

سپس ممکن است افزایش پیدا کند.



انتخاب بهترین مقدار `max_depth`

بر اساس تحلیل نمودارهای `accuracy` و `misclassification error`، مقدار بهینه `max_depth` که بهترین عملکرد را در داده‌های `train` و `validation` داشت، انتخاب شد. این مقدار برای آموزش مدل نهایی و پیش‌بینی داده‌های `test` استفاده شد.

ارزیابی نهایی مدل روی داده‌های `Test`

برای ارزیابی نهایی، تمام معیارهای ارزیابی `accuracy`, `precision`, `recall`, `F1-score` و `confusion matrix` روی داده‌های `test` محاسبه شد.

۱-۴ چگونه افزایش عمق درخت موجب افزایش variance میشود؟

درخت تصمیم یک مدل غیرخطی است و با افزایش max_depth، مدل می‌تواند پیچیدگی‌های بیشتری از داده‌ها را یاد بگیرد. یعنی با افزایش عمق، درخت تصمیم به واحدهای کوچکتر و جزئی‌تر تقسیم می‌شود، و به همین دلیل variance افزایش می‌یابد. این به مدل این امکان را می‌دهد که دقیقاً بر روی داده‌های آموزشی یاد بگیرد، اما در مقابل به داده‌های جدید و ناشناس حساس‌تر خواهد بود. این رفتار منجر به overfitting می‌شود.

۲-۴ در نمودار train/validation چه نشانه‌هایی از overfitting و underfitting قابل مشاهده است؟ این دو

مورد را روی نمودار مشخص نموده و با نمودار گام پیشین (k-NN) مقایسه نمایید؟

Underfitting -- زمانی که accuracy روی داده‌های train و validation بسیار پایین است خصوصاً برای k کوچک یا عمق کم درخت. این نشان‌دهنده این است که مدل قادر به یادگیری الگوهای پیچیده داده‌ها نیست.

Overfitting -- زمانی که accuracy روی داده‌های train بسیار بالا باشد، اما روی validation کاهش پیدا کند. این نشان‌دهنده این است که مدل به داده‌های آموزشی بیش از حد حساس شده و به generalization (تعمیم) ضعیف است.

در k-NN، به‌ویژه با k کوچک، شاهد overfitting شدید هستیم، که مشابه با عمق کم درخت تصمیم است. در درخت تصمیم، با افزایش عمق درخت، به تدریج variance افزایش می‌یابد و احتمال overfitting نیز بیشتر می‌شود.

۳-۴ هرس کردن درخت چه تأثیری روی مدل از نظر overfitting و underfitting دارد؟

هرس کردن درخت یعنی جلوگیری از بزرگ شدن بیش از حد درخت و حذف شاخه‌هایی که خیلی ریز و جزئی هستند. وقتی درخت خیلی عمیق می‌شود، معمولاً داده‌های train را حفظ می‌کند و به نوبه حساس می‌شود overfitting با هرس کردن:

درخت ساده‌تر می‌شود -- به جزئیات بی‌اهمیت توجه نمی‌کند -- روی داده‌های جدید عملکرد بهتری دارد. هرس کردن سبب کاهش overfitting -- variance کمتر می‌شود -- مدل قابلیت تعمیم (generalization) بهتری پیدا می‌کند

البته اگر هرس کردن بیش از حد انجام شود:

درخت خیلی ساده می‌شود / ممکن است underfitting اتفاق بیفتد
بنابراین، هرس کردن خوب باید متعادل باشد؛ نه خیلی زیاد، نه خیلی کم.

گام پنجم – پیاده سازی مدل LDA

Linear Discriminant Analysis (LDA) برای طبقه‌بندی داده‌ها استفاده شد. این مدل یک روش خطی و پارامتریک است که تلاش می‌کند با بیشینه کردن فاصله بین کلاس‌ها و کمینه کردن پراکندگی درون کلاسی، مرز تصمیم مناسبی ایجاد کند.

تحلیل ظرفیت مدل و رفتار Bias-Variance

این مدل برخلاف knn و درخت تصمیم که میتواند مرزهای پیچیده و غیرخطی بسازد. این مدل قادر به یادگیری مرزهای خطی میباشد. به همین دلیل Bias بالا چونکه مدل خطی است، نمی‌تواند پیچیدگی‌های داده را به درستی مدل کند. و Variance پایین میباشد چونکه ساده بودن مدل، حساسیت زیادی به داده‌ها ندارد و بیشتر برای پیش‌بینی‌های عمومی و ساده مناسب است. این دو ویژگی باعث می‌شوند که مدل LDA پایدار و کم‌نوسان باشد، ولی از طرف دیگر توانایی یادگیری روابط پیچیده‌تر را نداشته باشد.

۱-۵ چرا انتخاب نهایی مدل فقط باید بر اساس validation انجام شود؟

چون داده‌های test باید کاملاً شبیه داده‌های واقعی و دیده نشده باشند. اگر از test برای تنظیم مدل یا انتخاب نهایی استفاده کنیم، اطلاعات آن به مدل نشت می‌کند (Data Leakage) و ارزیابی نهایی غیرواقعی می‌شود.

۲-۵ چرا LDA معمولاً bias بیشتری نسبت به درخت تصمیم و k-NN دارد؟

چون LDA فرض می‌کند مرز تصمیم خطی است و شکل توزیع داده‌ها را محدود می‌کند و انعطاف‌پذیری کمتری نسبت به مدل‌های غیرپارامتریک دارد
در مقابل:

k-NN و درخت تصمیم می‌توانند مرزهای پیچیده و غیرخطی بسازند پس بنابراین bias کمتری دارند، اما معمولاً variance بیشتری دارند.

۳-۵ با توجه به دقت LDA روی train و validation، مدل بیشتر underfitting دارد یا overfitting؟

همانطور که مشاهده کردیم دقت train و validation به هم نزدیک هست و اختلاف زیادی بین این دو مشاهده نمی‌شود به همین دلیل می‌توان گفت که مدل overfitting ندارد. اما به دلیل خطی بودن مدل و محدودیت ظرفیت آن، احتمال دارد که LDA نتواند تمام پیچیدگی‌های داده را یاد بگیرد. به همین دلیل مدل LDA بیشتر تمایل به underfitting خفیف دارد

گام ششم – جمع بندی و مقایسه نهایی

مدل	Accuracy (Train)	Accuracy (Test)	Macro Precision (Train)	Macro Precision (Test)	Macro Recall (Train)	Macro Recall (Test)	Macro F1 (Train)	Macro F1 (Test)
k-NN	۰.۹۰۱۶	۰.۹۰۱۶	۰.۹۰۰۶	۰.۹۰۰۶	۰.۹۰۱۴	۰.۹۰۱۴	۰.۸۹۹۷	۰.۸۹۹۷
Decision Tree	۰.۹۸۰۱	۰.۸۱۹۸	۰.۹۸۰۲	۰.۸۱۷۵	۰.۹۸۰۱	۰.۸۱۹۸	۰.۹۸۰۰	۰.۸۱۷۹
LDA	۰.۶۴۹۱	۰.۶۴۰۶	۰.۶۵۶۹	۰.۶۴۹۷	۰.۶۴۸۹	۰.۶۴۰۶	۰.۶۵۱۹	۰.۶۴۴۰

مدل	پارامتر نهایی	ظرفیت مدل	Bias	Variance	تفسیرپذیری
k-NN	$k \approx 5-10$	زیاد	کم	متوسط	کم
Decision Tree	$\text{max_depth} \approx 8$	متوسط	متوسط	متوسط	زیاد
LDA	—	کم	زیاد	کم	متوسط

k-NN = دقت بالا، نوسان بیشتر

Decision Tree = بهترین تعادل + تفسیرپذیر

LDA = ساده، پایدار، اما کم دقت

تفسیر پذیری	Bias	Variance	ظرفیت مدل	پارامتر	مدل
خیلی کم	خیلی کم	خیلی زیاد	خیلی زیاد	k=1	k-NN
خیلی کم	کم	زیاد	زیاد	k=3	k-NN
کم	متوسط	متوسط	متوسط	k=5	k-NN
کم	متوسط	کم	کم	k=10	k-NN
کم	زیاد	خیلی کم	خیلی کم	k=20	k-NN
کم	خیلی زیاد	خیلی کم	خیلی کم	k=50	k-NN
زیاد	زیاد	کم	کم	max_depth=2	Decision Tree
زیاد	متوسط	متوسط	متوسط	max_depth=4	Decision Tree
زیاد	کم	زیاد	زیاد	max_depth=8	Decision Tree
متوسط	خیلی کم	خیلی زیاد	خیلی زیاد	max_depth=12	Decision Tree
کم	خیلی کم	خیلی زیاد	خیلی زیاد	max_depth=20	Decision Tree
متوسط	زیاد	کم	کم	—	LDA

تحلیل تفسیر پذیری مدل‌ها

: Decision Tree

مسیر تصمیم مشخص / قابل رسم و توضیح / بالاترین تفسیر پذیری

: LDA

ضرایب خطی قابل بررسی / تفسیر پذیری متوسط

: k-NN

تصمیم‌گیری بر اساس فاصله / بدون مدل صریح / کمترین تفسیر پذیری

مقایسه کلی مدل‌ها

k-NN

انعطاف‌پذیر / حساس به نویز / هزینه محاسباتی بالا / مناسب داده‌های ساده‌تر

Decision Tree

تفسیرپذیر / مستعد overfitting / با تنظیم عمق عملکرد مناسب

LDA

پایدار / bias بالا / مناسب زمانی که مرز تصمیم تقریباً خطی است

به طور کلی میتوان گفت که :

k-NN بهترین تعادل بین دقت، پایداری و توان تعمیم را در این پروژه داشته است، در حالی که درخت تصمیم دچار

overfitting و مدل LDA به دلیل خطی بودن دچار bias بالا شده است.